

NATIONAL CYBER LEAGUE CTF Guide

Prepared by:
Gabrielle Decker



OKLAHOMA CITY
COMMUNITY COLLEGE

🧠 CTF Prep Guide: Open Source Intelligence (OSINT)

This guide is designed for beginners, especially those brand new to NCL. It will walk you through the most common tools, tactics, and workflows you'll use to solve OSINT challenges — from finding hidden data in images to researching threat groups and infrastructure.

🎯 What Is OSINT?

Open Source Intelligence (OSINT) is information gathered from publicly available sources like websites, images, databases, and social media. In CTFs, OSINT challenges often ask you to extract hidden data, track locations, identify technology or people, or correlate technical indicators with public info.

◆ EXIF & Metadata Parsing

Example Challenges: `Meta.jpg`, `Target Building`, `Alley`

Goal: Extract image metadata to find timestamps, GPS coordinates, and camera details.

Tools:

- `exiftool <filename>` – Reads metadata
- GIMP or Photoshop – Useful for inspecting image layers visually

Look For:

- `CreateDate` → Round to the nearest minute
 - `GPSLatitude`, `GPSLongitude` → Paste into Google Maps
 - `Camera Make` / `Model` → Might hint at a location or device
-

◆ Geolocation & Landmark Recon

Example Challenges: Vantage Point , District , Geolocation , Tracking

Goal: Match locations using visual clues or GPS data.

Tools:

- Google Maps / Street View / Earth
- Satellite view to match rooftops, road curves, shadows

Tips:

- Use historical view if modern images don't match
- Compare signs, tree shapes, terrain to photo clues
- Paste coordinates directly into Google to start

◆ Internet Search & Passive Recon

Example Challenges: Threat Intel , Internet Lookup , Defense Acquisition

Goal: Identify info about a domain, org, person, or IP address.

Tools:

- whois , crt.sh , shodan.io , Wayback Machine
- ASN lookup tools: BGPView , ARIN

Tips:

- Use Google dorks: site:example.com ext:pdf
- Look up ASN ranges or domain certs on crt.sh
- Use whois for IP ranges and domain ownership

◆ Credential & Breach Analysis

Example Challenges: Breached Passwords , Compromise Report

Goal: Identify leaked or reused passwords, malware indicators, or exploit info.

Tools:

- Google (yes, really!)
- HaveIBeenPwned
- CVE databases (e.g., NVD, CVE Details)

Tips:

- Memorize known breach passwords (e.g., admin , solarwinds123 , fortinet backdoor)
 - Know infamous backdoors (e.g., VSFTPD smiley face)
 - Check DLLs, registry keys, or CVEs mentioned
-

- ◆ QR Codes, Barcodes, & Obscure Encodings

Example Challenges: Barcode , Message in Stone , Network Info

Goal: Extract text or flags hidden in visual codes or strange strings.

Tools:

- zbarimg file.png – Scan QR/barcodes
- ZXing Online Decoder
- CyberChef – Swiss army knife for encoding/decoding

Tips:

- Look for patterns like WIFI:S:SSID;T:WPA;P:password;;
 - Decode multiple times if needed (e.g., base64 → hex → plain text)
 - Don't forget to check rotated images/barcodes
-

◆ Social Media & Org Tracing

Example Challenges: Lookup , Trivia , APT , Restaurant WiFi , Codenames

Goal: Identify accounts, agencies, or people behind usernames or aliases.

Tools:

- Twitter, LinkedIn, Reddit, Crunchbase
- GovTribe, SAM.gov, FCC license databases

Tips:

- Cross-check org aliases (e.g., VCA = CIA)
- Search past NCL leaderboards or Reddit posts
- Use `site:linkedin.com "username"` style queries

Tool Summary

Tool	Purpose
exiftool	Image metadata (timestamps, GPS, camera)
zbarimg	QR and barcode decoding
CyberChef	Base64, hex, binary, ROT13, XOR, etc.
Google Maps	GPS coordinate resolution
whois , crt.sh , shodan	Passive DNS/domain/IP data
Wayback Machine	View old websites/pages
BGPView , ARIN	ASN, IP range lookups

Beginner Workflow Checklist

1. **Given an image?** → Start with `exiftool` → check for GPS or camera.
2. **Given coordinates?** → Paste into Google Maps → switch to Street View.

3. **Given usernames/handles?** → Try social media + Google.
 4. **Given domains/IPs?** → Use `whois`, `crt.sh`, `shodan.io`.
 5. **Given weird strings?** → Try CyberChef, ZXing, or `zbarimg`.
-

Final Advice

- Don't overthink early on — try basic tools first.
- Build your own cheat sheet from challenges you solve.
- If something doesn't work, try viewing it a different way: visually, reversed, encoded.
- Ask yourself: *what is this file, what is it showing me, and what could be hiding inside it?*

You got this. The web is your playground. 

CTF Prep Guide: Cryptography & Steganography

This guide is made for absolute beginners tackling cryptography challenges in the National Cyber League (NCL). You'll learn how to recognize encoded text, break classical ciphers, extract hidden data from files, and decode strange formats using simple tools and logic.

Section 1: Encoding & Base Conversions

What you're solving: Data encoded in formats like hexadecimal, base64, binary, or ASCII numbers.

Common patterns:

- `0x73636f727069666e` → hex → "scorpion"
- `c2NyaWJibGU=` → base64 → "scribble"
- `01110011 ...` → binary → text
- `115 107 121` → ASCII decimal → "sky"
- Chained encoding (e.g. base64 → hex → ASCII)

Tools:

- CyberChef : Drag-and-drop decoder
- Custom script: `base_converter.py`
- Terminal: `xxd` , `base64 -d` , `echo -e`

Tips:

- Try decoding with CyberChef first. It auto-detects common encodings.
 - Strip prefixes like `0x` before decoding.
 - Decode binary 8 bits at a time (1 byte).
-

Section 2: Classical Ciphers

What you're solving: Ciphers that replace letters or shift them based on rules.

Common patterns:

- **ROT13:** Letters rotated by 13 (e.g. `iveghny ynxr` → "virtual lake")
- **Caesar:** Shift letters (e.g. `hzuvob` → "email")
- **Atbash:** A↔Z, B↔Y, etc.
- **Vigenère:** Uses a keyword to shift letters
- **Polybius:** Grid coordinates to letters (e.g. `54 33 65...`)
- **T9 Keypad:** Encoded via phone keypresses (`7777` = "S")

Tools:

- dcode.fr for classic ciphers
- `caesar_decoder.py` (simple Caesar brute-force)
- `pycipher` (Python Vigenère solver)

Tips:

- Look for context clues (e.g., must start with "SKY")
- Try different tools for substitution-style ciphers

 Section 3: Obscure Formats & Hidden Patterns

What you're solving: Weird representations like T9, letter numbers, or disguised text.

Examples:

- T9 Keypad: `8-44-33` → letters via phone keypad
- Polybius Grid: Coordinate-based decryption
- Custom/emoji or symbolic patterns (e.g., alien-looking text)
- PGP encrypted messages (require private key)

Tools:

- CyberChef (great for chains)
- Paper charts for T9 or Polybius
- `gpg` or Kleopatra for PGP challenges

Tips:

- Try reversing known encodings
 - Copy strange symbols into CyberChef or text decoders
-

Section 4: Steganography

What you're solving: Hidden data inside image, audio, or other files.

Common patterns:

- Hidden text in PNGs using LSB (Least Significant Bit)
- ZIP files embedded in images
- Morse code/SSTV/DTMF in audio files
- Low contrast text in image (visual stego)

Tools:

- `stegseek`, `zsteg`, `binwalk`, `exiftool`
- Audacity, Sonic Visualiser, SSTV decoders
- `stegsolve.jar` (or any contrast/image plane viewer)

Tips:

- Run `binwalk` on images: `binwalk -e image.png`
 - Try `strings` to extract any embedded plain text
 - Open audio in spectrogram view to reveal patterns
-

Section 5: Advanced Ciphers & Crypto Primitives

What you're solving: Real encryption or pseudo-algorithms that require more than substitution.

Challenge types:

- XOR encryption (e.g., single-byte XOR)
- AES/DES with known key or IV hints
- Autokey XOR (where ciphertext is key for next round)
- DES IV reconstruction (partial IV given)

Tools:

- `CyberChef` : XOR brute-force, DES decryption
- `pycryptodome` (Python crypto library)
- `hashcat` with modes like `-m 12000` (XOR)

Tips:

- Look for key/IV hints in challenge text
- For XOR: test small brute-force scripts on 1-byte keys
- Use CyberChef to test theories quickly before scripting

 Tool Summary

Tool	Purpose
<code>CyberChef</code>	Encoding, cipher cracking, XOR, visual decoders
<code>base_converter.py</code>	Multi-decoder for base64, hex, bin
<code>dcode.fr</code>	Online solvers for classical ciphers
<code>stegseek</code> , <code>zsteg</code> , <code>binwalk</code>	Stego extraction from images
<code>Audacity</code> , <code>Sonic Visualiser</code>	Analyze spectrogram/audio stego
<code>gpg</code>	Decrypt PGP challenges with provided keypairs
<code>pycryptodome</code>	Script real crypto like AES/DES decryption

✅ Beginner Workflow Checklist

1. Got a weird string?

- Try base64 or hex decoding in CyberChef
- Use the auto-converter script if unsure

2. Got a scrambled sentence?

- Try ROT13, Caesar, or Vigenère on dcode.fr

3. Got a file (image/audio)?

- Run `binwalk`, `stegseek`, or `strings` on it
- Open it in a spectrogram tool if it's audio

4. Got a numeric pattern or code grid?

- Check if it matches T9, Polybius, or keypad layout

5. Got a .enc file or encrypted message?

- Check for hints: IV, key, or known format (PGP, DES)
- Try brute-forcing simple XOR keys with CyberChef

🧠 Final Advice

- Cryptography isn't about brute force — it's about recognizing patterns.
- You don't need to memorize every cipher — just know how to **identify** them.
- Learn to chain tools together (e.g., `binwalk` → `stegseek` → `CyberChef`).
- Write down working decoders/scripts and reuse them.

Remember, most crypto challenges hide their secrets in plain sight. You're not just solving a puzzle — you're learning how to think like a decoder. 🌱

CTF Prep Guide: Password Cracking

This guide helps absolute beginners get started with password cracking challenges in the National Cyber League (NCL). You'll learn how to identify hash types, crack passwords using dictionaries or custom patterns, and automate workflows using simple tools.

What is Password Cracking?

In CTFs, password cracking is the process of figuring out plaintext passwords from their hashed (encrypted) forms. This often involves:

- Recognizing hash types (e.g. MD5, SHA1, SHA256)
- Using tools to test large wordlists
- Building custom patterns for targeted guessing

You'll mostly be working with hash dumps, config files, WiFi keys, or password-protected documents.

Section 1: Hash Cracking with Wordlists

What you're solving: Passwords hashed with common algorithms.

Tools:

- `hashcat` – Fast GPU-based cracker
- `john` – CPU-based, great for NTLM/quick runs
- `rockyou.txt` – Popular wordlist of leaked passwords

How to use:

```
# Basic MD5 cracking
hashcat -m 0 hashes.txt rockyou.txt
```

```
# John with rockyou
john --wordlist=rockyou.txt hashes.txt
```

Tips:

- Always identify the hash type first (see next section)
 - Try rockyou.txt first — it often gets quick wins
-

Section 2: Salted Hashes & Format Detection

What you're solving: Hashes with salt values (extra random strings).

Common formats:

- `user:hash:salt`
- `user:hash1:hash2` (e.g., NTLM)

How to crack:

```
# Hashcat with username-aware salted hashes
hashcat -m 20 -a 0 salted.txt rockyou.txt --username

# NTLM example (Windows passwords)
hashcat -m 1000 -a 0 ntlm.txt rockyou.txt
```

Tip: Use `hashid` or `hash-identifier` to guess hash type:

```
hashid hashes.txt
```

Section 3: Custom Format & Pattern Cracking

What you're solving: Passwords that follow a known structure.

Examples:

- SKY-HQNT-####
- adjective+noun+digits (e.g., *sleepycat12*)
- PokemonName , TVShowName01

Tools:

- hashcat mask mode (`-a 3`)
- crunch , maskprocessor , or simple Bash loops

Examples:

```
# Hashcat mask attack for SKY-HQNT-1234
hashcat -a 3 -m 0 hashes.txt SKY-HQNT-?d?d?d?d

# Combo generator for themed cracking
combinator.py adjectives.txt nouns.txt > base.txt
crunch 2 2 -t @@ > digits.txt
for b in $(cat base.txt); do for d in $(cat digits.txt); do echo $b$d; done;
done > final.txt
```

Section 4: Known Plaintext / Hash Matching

What you're solving: You're given the plaintext and need to match the hash.

Example:

```
echo -n 'password' | md5sum
echo -n 'secret' | sha1sum
echo -n 'flag' | sha256sum
```

Tools:

- Terminal: md5sum , sha1sum , sha256sum
- CyberChef for quick copy/paste encoding chains

Section 5: WiFi Password Cracking

You might get:

- `wpa_supplicant.conf` with `psk=` lines
- `.cap` capture files with WPA handshakes

Tools:

- `aircrack-ng` or `pyrit`

Examples:

```
# Crack handshake with wordlist
aircrack-ng handshake.cap -w rockyou.txt

# Config file: look for psk="password" or hex
cat wpa_supplicant.conf
```

Section 6: Document/Archive Cracking

What you're solving: Password-protected `.pptx` , `.zip` , `.docx` , etc.

Tools:

- `msoffcrypto-crack` – Office files
- `fcrackzip` – Encrypted zip archives

Examples:

```
# Crack encrypted PPTX
msoffcrypto-crack -d rockyou.txt -o decrypted.pptx encrypted.pptx
```

```
# Crack zip archive
fcrackzip -u -D -p rockyou.txt file.zip
```

 Section 7: Themed Passwords & Wordlist Building

Examples:

- TV shows (Law & Order, Pokemon)
- Common adjective+noun combos

Build your own list:

```
cat shows.txt | tr '[:upper:]' '[:lower:]' | sed 's/ /_/g' > theme.txt
```

Use suffixes like 01, 12, 99:

```
sed 's/$/01/' theme.txt > final_list.txt
```

🔧 Tool Quick Reference

Tool	Use Case
hashcat	GPU hash cracking
john	CPU-based fast cracking
hashid	Detect hash type
rockyou.txt	Popular password wordlist
aircrack-ng	WPA handshake cracking
msoffcrypto-crack	Office file password cracking
fcrackzip	Zip archive password cracking
maskprocessor	Generate masks for Hashcat
combinator.py	Merge wordlists for hybrid attacks

✅ Beginner Workflow Checklist

1. **Is it a simple hash?** Try `hashid`, then:

```
hashcat -m 0 hash.txt rockyou.txt
```

2. **Is it salted (colon-separated)?** Try with `--username` or split it.

3. **Is it themed?** Build a wordlist with adjectives, Pokemon, TV shows.

4. **Is it WPA WiFi?** Use `aircrack-ng` and a capture file.

5. **Is it a document or ZIP file?** Use `msoffcrypto-crack` or `fcrackzip`.

6. **Document cracked results.** Keep a text file of what worked.

🧠 Final Advice

- Always start with wordlists — 80% of easy wins are in `rockyou.txt`
- When stuck, test popular hash modes: `0`, `100`, `1000`, `1400`
- Automate hash matching with CyberChef or bash one-liners
- Build a process you can reuse (mask, wordlist, script, repeat)
- Cracking isn't magic — it's patterns + tooling + persistence

The best crack isn't the fastest. It's the one that works. ⚡

CTF Prep Guide: Log Analysis

This guide is designed for beginners in the National Cyber League (NCL) and introduces you to the basics of identifying and analyzing log files in CTF challenges. You'll learn how to parse logs, recognize formats, and extract useful information quickly using command-line tools.

What is Log Analysis?

Log analysis is the process of reviewing and interpreting machine-generated logs to detect patterns, anomalies, or important details. NCL challenges test your ability to spot signs of attacks, behaviors, or system activity.

◆ Log Types You Might See

Log Type	File Example	What You'll Do
SSHAuth Logs	<code>auth.log</code>	Detect brute-force attempts and successful logins
Antivirus Logs	<code>Virus Scan.log</code>	Identify OS info, drives, RAM, and manufacturer
DNS Logs	<code>pihole.log</code>	Analyze domains, clients, and request types
Web Access Logs	<code>access.log</code>	Filter HTTP codes, user agents, and endpoints
Error Logs	<code>error.log</code>	Spot 404s, attackers, and misconfig paths
Browser History (SQLite)	<code>browser.sqlite</code>	Query for visited URLs, Bitcoin activity, logins
Serialized Logs	<code>lunch.log</code>	Parse weird formats like key=value or custom blobs
IDS Alerts	<code>eve_alert.json</code>	Analyze JSON alerts from intrusion detection systems
Physical Logs	<code>keyCard.log</code> + <code>motion.log</code>	Correlate time-based events for tampering detection
Proxy Logs	<code>squid_access.log</code>	Check traffic patterns, antivirus updates, GET/POST

📁 Basic Tools You Should Know

Tool	Purpose
<code>grep</code> , <code>cut</code> , <code>awk</code>	Text filtering and field extraction
<code>wc -l</code> , <code>uniq -c</code>	Counting lines, IPs, or errors
<code>jq</code>	Parse and filter JSON logs like <code>eve_alert.json</code>
<code>sqlite3</code>	Query SQLite databases (e.g., browser history)
Python + pandas	Custom parsing and reporting scripts

🔍 How to Identify a Log Type

When you're handed a file and you're not sure what it is, try this logic:

- Contains `sshd[...]` → likely `auth.log`
- JSON with `"alert"` → `eve_alert.json`
- Lots of `GET / POST` + status codes → `access.log`
- Repeated `File does not exist` → `error.log`
- Lines start with timestamps and latency → `squid_access.log`
- Binary starts with `SQLite format 3` → `browser.sqlite`
- Weird fields like `A=`, `C=`, `X=`, etc. → custom or `lunch.log`

Bonus: You can build a Python script (`logid.py`) to auto-detect types!

⚡ Useful Command Examples

SSH Logs

```
grep 'Failed password' auth.log | wc -l
grep 'Accepted password' auth.log
```

Access Logs

```
awk '{print $9}' access.log | sort | uniq -c # Count status codes
awk '{print $1}' access.log | sort | uniq -c # Unique IPs
```

Error Logs

```
grep 'File does not exist' error.log | wc -l
```

IDS Alerts (Suricata)

```
jq '.alert.signature' eve_alert.json | sort | uniq -c | sort -nr
```

Browser History (SQLite)

```
SELECT url, title FROM moz_places ORDER BY visit_count DESC;
```

🔧 Automation Tip: Build `logid.py`

Make a script that checks for known keywords and tells you what log type you have. This saves time during the CTF.

🚀 Beginner Workflow Checklist

1. **Got a .log or .json file?** Use `less`, `cat`, or `head` to preview it.
 2. **Identify the log type.** Look for patterns or use your `logid` script.
 3. **Use CLI tools.** Use `grep`, `awk`, `jq`, or SQL to filter or count.
 4. **Summarize.** Figure out top IPs, error counts, or biggest spikes.
 5. **Correlate.** Compare timestamps between logs for overlap or anomalies.
-

🧐 Final Advice

- Don't try to read entire logs. Use search and count tools.
- Look for patterns: repeated IPs, suspicious status codes, or rare user agents.
- Focus on what the challenge asks: "Who did what, when, and how?"
- Build a mini cheat sheet of filters and SQL queries as you go.

You're not just reading logs — you're uncovering stories hidden in plain text. 🕵️

🧠 CTF Prep Guide: Network Traffic & Wireless Exploitation

This guide teaches you how to analyze `.pcap`, `.cap`, and `.pcapng` files to uncover flags by reconstructing traffic, extracting credentials, identifying protocol behavior, and detecting covert communications. It also includes how to crack WEP and WPA2 wireless captures. This guide is written for beginner and intermediate CTF participants looking to build universal workflows.

🎯 Core Challenge Types

Category	Description
Protocol Analysis	DNS, FTP, HTTP, Telnet, VPN encapsulation
Credential Extraction	From FTP, Telnet, HTTP headers or payloads
Traffic Reconstruction	Rebuilding sessions, downloaded files, user interactions
WPA/WEP Cracking	Cracking wireless passwords using captured handshakes or IVs
Deauthentication	Identifying DoS attacks on WiFi clients
Device Profiling	Vendor detection via MAC, traffic pattern, or headers
Custom Protocols	Byte-level reverse engineering in hex/structured streams
IDS Logs	Suricata/Snort alerts from JSON or log files
Covert Channels	Exfiltration via beacons, chunked payloads, tunnels

🔧 Tools You'll Use

Tool	Purpose
Wireshark	Visual protocol analyzer, stream following
tshark	CLI-based packet parsing
tcpdump, capinfos	CLI capture filtering and metadata
aircrack-ng	WEP/WPA2 handshake cracking
pyrit, cowpatty	WPA analysis and cracking tools
CyberChef	Decode hex, base64, chunked payloads

Tool	Purpose
jq	Filter <code>.json</code> logs (e.g., Suricata)
binwalk, foremost	Carve embedded files or payloads from sessions
scapy	Scripted packet parsing and protocol emulation
zbarimg, qrencode	Analyze QR codes embedded in network data

Network Forensics Workflow

1. Preview the Capture

- Use `capinfos file.pcap` to review packet count, duration, and protocols

2. Identify Protocols

- Use filters in Wireshark:
 - `ftp, telnet, http, dns, eapol, icmp`
 - Or port-based: `tcp.port == 80, udp.port == 53`

3. Credential Extraction

- Follow TCP stream for HTTP, FTP, or Telnet traffic
- Look for `USER`, `PASS`, or `Authorization:` headers
- HTTP: Examine login forms or POST requests

4. File Reconstruction

- Wireshark: `File → Export Objects → HTTP`
- Save raw data from stream → Decode in CyberChef (hex, base64, etc.)

5. DNS Query Analysis

- Filter: `dns.qry.name`
- Check for exfiltration via long subdomains or suspicious query patterns
- Analyze TTL values

6. VPN or Encrypted Channel Detection

- Look for IPSec, WireGuard, OpenVPN tunnels
- Compare pre/post-tunnel IPs
- Identify encapsulated TCP or DNS inside encrypted traffic

Wireless Exploitation Workflow

1. Identify Network Type

- Filter: `wlan.fc.type_subtype == 0x08` (beacons)
- Check SSID and encryption type (WEP/WPA2)

2. Crack WEP

```
aircrack-ng file.cap
```

- Check IV count: filter `wlan.wep.iv`

3. Crack WPA2

```
aircrack-ng -w rockyou.txt file.cap
```

- Filter for handshake: `eapol`

4. Detect Deauth Attacks

- Filter: `wlan.fc.type_subtype == 0x0c`
- Link source MAC to target MACs

5. MAC Vendor Fingerprinting

- Look up MACs with `macvendors.com` for device type/context

✖ Custom Protocol Reversing

- Use `tshark -x` or Hex view in Wireshark
- Look for structured headers, identifiers, length fields
- Simulate protocol logic in Python or with Scapy:

```
from scapy.all import *  
# Custom field parsing from raw byte stream
```

🇮🇹 IDS Log Parsing

- Suricata/Zeek-style logs (e.g., `eve_alert.json`):

```
jq '.alert.signature' eve_alert.json | sort | uniq -c | sort -nr
```

- Focus on:
 - Most common alerts
 - Non-TCP attacks (ICMP, DNS)
 - Total byte transfers to suspicious IPs
-

Covert Channels & Exfil Detection

- Look for:
 - Data embedded in HTTP chunked POSTs or long query strings
 - Beacon SSIDs with encoded strings
 - Suspicious DNS query structures
 - Decode with CyberChef or script decoding chains:
 - Base64 → Hex → ASCII
 - Binary or Morse → Text
-

Full Checklist

- ☐ Run `capinfos` or `tshark -z io,stat` for summary
 - ☐ Identify protocols: DNS, FTP, HTTP, VPN, eapol, beacon
 - ☐ Reconstruct login credentials via stream inspection
 - ☐ Export and decode downloaded files
 - ☐ Crack WEP/WPA2 handshakes with `aircrack-ng`
 - ☐ Check for deauth and beacon manipulation
 - ☐ Parse `.json` IDS logs with `jq` and filter by signature/IP
 - ☐ Profile MACs, IPs, and traffic flow per device
 - ☐ Test for covert or encoded traffic (WiFi SSIDs, subdomains, payloads)
-

Final Advice

- Every `.pcap` is a mystery with a timeline. Focus on who talks to who, how, and why.

- Decode everything that looks too long, too random, or too well-formed.
- Most flags aren't hidden — they're transmitted. You just have to catch them in the act.

Packets don't lie. Flags hide in flow. Let the traffic talk.

CTF Prep Guide: Digital Forensics

This universal guide is for beginners learning how to approach digital forensics challenges in Capture the Flag (CTF) environments. You'll learn how to extract hidden files, analyze metadata, carve data from binaries, and investigate memory dumps using essential tools and workflows.

Core Forensics Categories

Forensic Type	Description
Image Analysis	Detect hidden messages or metadata in photos/images
File Carving	Extract files embedded in unknown binary blobs
Archive Recovery	Explore <code>.tar</code> , <code>.zip</code> , or <code>.7z</code> for secrets and structure
Git & Versioning	Investigate <code>.git</code> folders for exposed or deleted content
Memory Dump Triage	Recover processes, file paths, and flags from RAM snapshots

Essential Tools & Utilities

Image & Metadata Forensics

- `exiftool` – Extract timestamps, GPS coordinates, and camera info
- `binwalk -e` – Carve embedded files from image or binary blobs
- `zsteg`, `steghide` – Detect steganography in images
- `strings`, `xxd`, `hexdump` – Discover readable content in files

Archive & File Parsing

- `tar`, `unzip`, `7z` – Extract archive contents
- `jq` – Pretty-print or parse JSON
- `sqlite3` – Explore `.sqlite` databases
- `file`, `xxd` – Identify and inspect unknown file types

Memory Analysis

- `volatility3` – Analyze Linux memory dumps with plugins like:
 - `linux_pslist`, `linux_find_file`, `linux_bash`, `linux_proc_maps`
 - `strings` – Search memory dumps for readable keywords like `flag`
-

Common Analysis Workflows

Image/Metadata Analysis

```
exiftool file.png          # Metadata
binwalk -e file.png        # Extract embedded data
zsteg file.png             # LSB stego (PNG only)
```

Carving from Binary Blobs

```
xxd unknown_blob | head   # Check for magic byte headers
binwalk -e unknown_blob   # Carve known files automatically
```

Archive/JSON/SQLite Parsing

```
tar -xvf archive.tar
jq '.' config.json          # Pretty-print JSON
sqlite3 file.sqlite
> .tables
> SELECT * FROM some_table;
```

Memory Dump Triage

```
xz -d dump.mem.xz          # Decompress if needed
vol.py -f dump.mem linux_pslist # Process list
vol.py -f dump.mem linux_find_file -i flag
strings dump.mem | grep -i flag
```

Recover deleted files in Git repositories:

```
git checkout $(git rev-list --all) -- <filename>
```

Search for sensitive fields in JSON:

```
jq '["secret"]' config.json
```

Identify EXIF metadata fields:

```
exiftool file.jpg | grep -i 'gps\|date\|model\|make'
```

Universal Workflow Checklist

1. **Got an image?** Run `exiftool`, `binwalk`, `zsteg`, and view it visually
2. **Got a blob?** Try `xxd`, then run `binwalk` to look for file signatures
3. **Got an archive?** Extract contents → check for `.json`, `.sqlite`, `.env`, etc.
4. **Got memory?** Decompress and run `volatility3` plugins + `strings`
5. **Collect artifacts:** Build a folder of extracted files, flags, and credentials

Final Advice

- Start broad: check structure and metadata first
- Let automated tools carve and extract before going manual
- Be ready to switch tools: images, configs, and memory require different approaches
- Most flags hide in overlooked places: configs, version control, temp files

In forensics, what others miss is your opportunity. Be thorough. Be curious.

CTF Prep Guide: Scanning & Reconnaissance

This universal guide is for beginners practicing network enumeration and reconnaissance in Capture the Flag (CTF) environments. These skills help you discover open services, hidden files, and unprotected endpoints—essential for early-stage footprinting and exploitation.

Core Skills You'll Develop

Category	Description
Port Scanning	Identify open ports (TCP/UDP) and running services
Service Discovery	Detect what software is running on which ports
Web Enumeration	Discover hidden directories or files on websites
Decentralized Recon	Query services like IPFS or P2P trackers
Manual Probing	Use low-level tools to interact with services

Essential Tools

Tool/Command	What It Does
<code>nmap</code>	Scan ports, detect versions, enumerate services
<code>ffuf</code> , <code>gobuster</code> , <code>dirb</code>	Brute-force web file and directory discovery
<code>curl</code> , <code>nc</code> , <code>openssl</code>	Manually interact with HTTP, raw TCP, TLS
<code>whatweb</code> , <code>httprobe</code>	Detect HTTP software and status
<code>ipfs</code> , <code>torrent clients</code>	Access decentralized data (if applicable)
<code>CyberChef</code> , <code>xxd</code>	Decode, reverse, or hex-decode artifacts

Universal Workflows

Port Scanning

```
nmap -sS -sV -p- <target>      # Full TCP scan with version detection
nmap -sU -p- <target>          # Full UDP scan
nmap -sC -sV -T4 -A <target>   # Aggressive recon with scripts
```

Web Enumeration

```
ffuf -u https://site.com/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

- Look for flags in common names like `flag.txt`, `hidden`, `backup/`, `.git/`
- View page source, comments, robots.txt, and `sitemap.xml`

Manual Service Fingerprinting

```
curl -I http://target          # See server header
nc target 80                   # Try manual request like GET / HTTP/1.1
openssl s_client -connect target:443 # Inspect TLS certs
```

Decentralized/Obscure Protocols

- Look for content hashes (e.g., IPFS: starts with `Qm...`)
- Look for tracker patterns (e.g., BitTorrent-style `.torrent` hashes)

```
echo "466c6167..." | xxd -r -p      # Decode hex flag manually
```

Service Discovery Tips

- For HTTP(S), start with `curl`, `gobuster`, or browser + dev tools
 - For unknown ports, connect with `nc` or `nmap -sV`
 - Look for default credentials, exposed backups, `.env`, and misconfigurations
-

✓ Beginner Recon Checklist

1. **Start with Nmap:** TCP, then optionally UDP
 2. **See an open HTTP port?** Try `curl`, browser, or run `ffuf`
 3. **Got a weird hash or port?** Use CyberChef or analyze manually
 4. **Check services with Netcat/OpenSSL:** Probe for readable responses
 5. **Document everything:** Ports, software, versions, URLs, flags
-

🧠 Final Advice

- Don't start with brute-force. Start with observation.
- Think like a developer: what might be accidentally exposed?
- Scan, fingerprint, enumerate, and *then* test for vulnerabilities
- Recon is 80% of the battle — know the terrain, and you'll find the win path

Recon is the map that leads to the treasure. Learn to read it.

CTF Prep Guide: Web Application Exploitation

This guide is a universal resource for tackling web app vulnerabilities in CTF challenges. It equips you with repeatable methods to test for authentication flaws, reverse JavaScript logic, explore APIs, and uncover hidden paths—using only beginner-friendly tools and tactics.

Core Vulnerability Categories

Category	What It Covers
Authentication Bypass	SQL injection, logic flaws, client-side form bypass
Session & Token Manipulation	Cookie tampering, JWT editing, localStorage attacks
IDOR / Authorization	Accessing unintended user data via ID changes
JavaScript Reversal	Revealing frontend flag logic and hidden API calls
File/Directory Discovery	Brute-force and path-guessing for hidden endpoints
Business Logic Exploits	Scheduling abuse, conditional flag reveals, loopholes

Tools You'll Use Often

Tool	Purpose
Browser DevTools	View network traffic, modify JS/HTML, test input logic
Burp Suite (Community)	Intercept, repeat, and modify HTTP traffic
ffuf , dirsearch	Discover hidden directories or backup files
curl , Postman	Replay and test raw HTTP/JSON requests
jwt.io , CyberChef	Decode, modify, or replay JWTs and base64 tokens

Universal Web Exploitation Flow

1. Start in DevTools:

- Open browser console and inspect:
 - `Network` tab for fetch/XHR
 - `Sources` tab for `.js` files
 - `Application` tab for cookies, storage

2. Look for Exposed Clues:

- Endpoints like `/api/`, `/flag`, `.env`, `.json`
- Visible JavaScript variables or `if(score >= ...)`

3. Token or Cookie Analysis:

- Decode JWTs
- Modify `role`, `id`, `auth` if present
- Watch how changing cookies affects access

4. Replay Requests:

- Copy `curl` from browser
- Use Burp/Postman to modify params or headers
- Try IDORs: switch `/user/123/` → `/user/1/`

5. Try Path Tricks:

- Manually test for `/admin`, `/backup.bak`, `/debug`, `/flag`
- Use `ffuf` only if allowed:

```
ffuf -u https://site.com/FUZZ -w wordlists/common.txt
```

6. Look for Logic Gaps:

- Can you replay a flag-granting action?
- Is the frontend gate enforced server-side?

Vulnerability Tactics

Authentication Bypass

- `' OR '1'='1` in login form fields
- Remove `disabled` attributes in DevTools
- Replay login request with changed credentials

Session & Role Tampering

- JWT token with editable fields: `{ "role": "user" }`
- Change to `admin` and replay requests
- Inspect cookies and localStorage in DevTools

JavaScript Reversal

- Find `.js` logic
- Look for hardcoded flags, token checks, or score thresholds
- Use breakpoints to halt execution before form submission

Directory & File Discovery

- Manually test:
 - `/flag`, `/flag.txt`, `/admin`, `/robots.txt`, `.bak`, `.old`
- Use automated tools when allowed (ffuf/dirsearch)

Business Logic Exploits

- Parameter tampering:

```
{ "action": "win", "slot": "2038-01-01T00:00Z" }
```

- Re-submit or replay requests that trigger flag drops

Web Exploitation Checklist

- ☐ Open DevTools → check Network, Sources, Application tabs for potential clues
- ☐ Inspect API calls and JavaScript behavior for access logic or score thresholds
- ☐ Locate `.js`, `.json`, `.env`, or `.bak` files manually or using path testing
- ☐ Modify JWTs, cookies, or localStorage values if client-side logic is suspected
- ☐ Replay API/login requests using curl, Burp Suite, or Postman with modified params
- ☐ Test URL paths for IDOR vulnerabilities or unsecured admin routes
- ☐ Look for exposed admin logic, user identifiers, or flag-triggering conditions
- ☐ Track and replay logic that mimics "win", "claim", or "admin" actions
- ☐ Open DevTools → check Network, Sources, Application

- ☐ Find `.js` or `.json` files → search for `flag`, `admin`, `score`
 - ☐ Replay login/API requests with Burp or curl
 - ☐ Try IDOR by switching numeric values in URL or POST
 - ☐ Decode and modify JWTs, then re-test access
 - ☐ Probe endpoints manually for file disclosures
 - ☐ Watch for form conditions: scores, win counts, click tracking
-

Final Advice

- Don't trust the frontend — DevTools is your map, Burp is your weapon
- Focus on how the app decides who gets the flag
- If something is hidden via JS, ask: *is it enforced server-side?*
- Reuse tokens, replay requests, and clone behavior
- Keep notes on every flag path — you'll see patterns repeat

Every flag was meant to be guarded. Your job is to prove it wasn't.

CTF Prep Guide: Enumeration & Exploitation

This guide helps you develop skills to analyze and reverse logic from compiled code, scripts, and binaries — commonly seen in reverse engineering (RE), logic puzzles, and exploitation challenges in CTFs. You'll learn how to decode obfuscation, simulate logic flows, and identify vulnerabilities in everything from Python bytecode to custom virtual machines.

Core Vulnerability Categories

Category	What It Involves
Python RE & Obfuscation	XOR decoding, reverse logic, <code>.pyc</code> decompilation
Go & Unusual Languages	Flow reconstruction, ASCII checks, channel/goroutine behavior
Binary Reversing	Disassembling ELF/PE files, locating hardcoded flags, branch tracing
Custom VM Reversing	Opcode logic emulation, manual instruction tracing
Hashing & Brute Simulations	Proof-of-work puzzles, hash manipulation, logic loop analysis

Essential Tools

Tool/Utility	Use Case
<code>Python</code> <code>shell</code>	Simulating logic, XOR decoding, scripting solutions
<code>CyberChef</code>	Visual XOR/brute-force manipulation, hex decoding
<code>uncompyle6</code> , <code>pycdc</code>	Decompiling <code>.pyc</code> Python bytecode files
<code>Ghidra</code> , <code>IDA Free</code>	Disassembling and analyzing binary executables (ELF/PE)
<code>strings</code> , <code>objdump</code>	Static inspection of binary files for flag clues
<code>xxd</code> , <code>hexdump</code>	Viewing raw bytecode or encoded structures
<code>ltrace</code> , <code>strace</code> , <code>gdb</code>	Runtime tracing and debugging for dynamic behavior analysis

1. Check the File Type:

- Run `file <filename>` to identify Python `.pyc`, ELF, or PE formats.

2. If Python Source or PYC:

- Try decompiling:

```
uncompyle6 file.pyc > decompiled.py
```

- Simulate conditions like XOR or `ord()`-based checks:

```
vals = [130, 154, 136, ...]  
decoded = "".join([chr(v ^ 209) for v in vals])  
print(decoded)
```

3. If It's a Compiled Binary:

- Start with:

```
strings binary | grep flag  
objdump -d binary | less
```

- Load in Ghidra or IDA and:
 - Identify `main()` function
 - Trace comparisons or validation logic (e.g., `strcmp`, `memcmp`)
 - Look for hardcoded flag strings or hash matches

4. If You Suspect a Custom VM or Opcode Logic:

- Use `xxd` to inspect the binary:

```
xxd binary | head
```

- Write a Python script to simulate:

```
with open("prog.bin", "rb") as f:  
    bytecode = f.read()  
for op in bytecode:  
    print(hex(op)) # Simulate decoding behavior
```

5. If You See a Proof-of-Work or Hash Puzzle:

- Identify the hash condition:

```
import hashlib
for i in range(1000000):
    h = hashlib.sha256(str(i).encode()).hexdigest()
    if h.startswith("0000"):
        print(i, h)
        break
```

Exploitation Techniques by Category

Python XOR / `ord()` Checks

- Often used to mask flags in Python logic.
- Reverse each byte with `chr(value ^ key)`.

Decompiled `.pyc` Files

- Use `uncompyle6` to restore source.
- Look for:
 - `if` conditions
 - Length checks
 - Sum or checksum comparisons

Binary Reversing (ELF/PE)

- Use `Ghidra` to view:
 - Control flow (branch conditions)
 - String references (`.rodata`)
 - Comparison operations

Custom VM Challenges

- Treat unknown binary blobs as virtual instruction sets.
- Look for repeating byte patterns (likely opcodes).
- Write small emulators to test behavior step-by-step.

Hashing / Mining

- Simulate loops that test hashes with `hashlib`.
 - Try brute-forcing input strings until hashes match target pattern (e.g., starts with `0000`).
-

Exploitation Checklist

- ☐ Identify the file type and extract key bytes or structure
 - ☐ If XOR or `ord()` patterns are present, reverse them with Python or CyberChef
 - ☐ Use `uncompyle6` or `pycdc` to decompile `.pyc` files
 - ☐ If binary, run `strings`, `objdump`, or load in Ghidra
 - ☐ If it's a VM, write a custom emulator or log opcodes
 - ☐ Simulate hash comparisons if a mining puzzle is present
 - ☐ Take notes on every branch, input condition, or decoded string
-

Final Advice

Enumeration & Exploitation is about **reading code like a story**. Every check, comparison, or encoded byte is a clue left by the author.

- Don't brute-force until you've traced the logic.
- Simulate what the program expects, then meet those expectations surgically.
- If stuck, ask: *What is this code really trying to prove true before it releases the flag?*
- Keep your tools handy, and don't be afraid to write throwaway scripts to test ideas.

These challenges reward logical precision, code empathy, and patience. The more you train your intuition for how logic is obfuscated, the more flags you'll recover by **thinking like the code**.